

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	6
1.2 Описание выходных данных.....	6
2 МЕТОД РЕШЕНИЯ.....	7
3 ОПИСАНИЕ АЛГОРИТМОВ.....	9
3.1 Алгоритм конструктора класса Device.....	9
3.2 Алгоритм метода show класса Device.....	9
3.3 Алгоритм метода showStats класса Device.....	10
3.4 Алгоритм деструктора класса Device.....	10
3.5 Алгоритм функции main.....	10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	12
5 КОД ПРОГРАММЫ.....	15
5.1 Файл Device.cpp.....	15
5.2 Файл Device.h.....	15
5.3 Файл main.cpp.....	16
6 ТЕСТИРОВАНИЕ.....	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	19

1 ПОСТАНОВКА ЗАДАЧИ

В информационной системе центра обработки данных ведётся учёт вычислительных устройств.

Каждое устройство имеет:

- имя устройства;
- уникальный номер устройства.

Кроме индивидуальных данных устройств, система хранит общую информацию:

- общее количество созданных устройств;
- текущее количество активных устройств.

Для реализации такой системы необходимо использовать статические поля класса.

Класс Устройство

Поля:

- строковое поле имени устройства;
- целочисленное поле уникального идентификатора устройства();
- общее количество созданных устройств;
- текущее количество активных устройств.

Методы:

- конструктор, сохраняющий имя устройства, увеличивающий счётчик созданных устройств, увеличивающий счётчик активных устройств, присваивающий уникальный номер устройства через счетчик созданных устройств.

- деструктор, уменьшающий число активных устройств;
- метод вывода данных объекта в формате "Device::show -> <name> <id>";

- статический метод вывода данных о созданных и активных объектах в формате "Device::showStats -> created: <value> active: <value>"

Алгоритм работы программы

1. Ввести три имени устройств.
2. Создать:
 - 2.1. один объект автоматически;
 - 2.2. два объекта динамически.
3. Для каждого объекта вывести установленные данные.
4. Вывести информацию о кол-ве созданных и активных устройств.
5. Удалить первый динамический объект.
6. Вывести информацию о кол-ве созданных и активных устройств.
7. Удалить второй динамический объект.
8. Завершить программу.

1.1 Описание входных данных

<name1> <name2> <name3>

1.2 Описание выходных данных

Device::show -> <name1> <id1>

Device::show -> <name2> <id2>

Device::show -> <name3> <id3>

Device::showStats -> created: <value> active: <value>

Device::showStats -> created: <value> active: <value>

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект dev1 класса Device предназначен для создания объекта класса;
- объект dev2 класса Device предназначен для создания объекта класса;
- объект dev3 класса Device предназначен для создания объекта класса;
- объект cin класса istream предназначен для ввода с клавиатуры.

Класс Device:

- свойства/поля:
 - поле имя устройства:
 - наименование — name;
 - тип — string;
 - модификатор доступа — private;
 - поле уникальный идентификатор устройства:
 - наименование — id;
 - тип — int;
 - модификатор доступа — private;
 - поле общее количество созданных устройств:
 - наименование — ndevices;
 - тип — static int;
 - модификатор доступа — private;
 - поле текущее количество активных устройств:
 - наименование — nactive;
 - тип — static int;

- модификатор доступа — private;
- функционал:
 - о метод Device — конструктор с параметром;
 - о метод show — метод вывода данных объекта;
 - о метод showStats — статический метод вывода данных о созданных и активных объектах;
 - о метод ~Device — деструктор.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Device

Функционал: конструктор с параметром.

Параметры: string name - имя устройства.

Алгоритм конструктора представлен в таблице 1.

Таблица 1 – Алгоритм конструктора класса Device

№	Предикат	Действия	№ перехода
1		присвоение закрытому свойству name значение параметра name	2
2		увеличение статического поля ndevices на 1	3
3		увеличение статического поля nactive на 1	4
4		присовение закрытому свойству id значение статического поля ndevices	Ø

3.2 Алгоритм метода show класса Device

Функционал: метод вывода данных объекта.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода show класса Device

№	Предикат	Действия	№ перехода
1		вывод на экран "Device::show -> <name> <id>"	Ø

3.3 Алгоритм метода showStats класса Device

Функционал: статический метод вывода данных о созданных и активных объектах.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода showStats класса Device

№	Предикат	Действия	№ перехода
1		вывод на экран "Device::showStats -> created: <value> active: <value>"	Ø

3.4 Алгоритм деструктора класса Device

Функционал: деструктор.

Параметры: нет.

Алгоритм деструктора представлен в таблице 4.

Таблица 4 – Алгоритм деструктора класса Device

№	Предикат	Действия	№ перехода
1		уменьшение статического поля на 1	Ø

3.5 Алгоритм функции main

Функционал: основной алгоритм программы.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 5.

Таблица 5 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявление переменных name1, name2, name3 типа string	2
2		ввод с клавиатуры переменных name1, name2, name3	3
3		объявление объекта dev1 с параметром name1	4
4		динамическое объявление объекта dev2 с параметром name2	5
5		динамическое объявление объекта dev3 с параметром name3	6
6		вызов метода show объекта dev1	7
7		вызов метода show объекта dev2	8
8		вызов метода show объекта dev3	9
9		вызов статического метода showStats	10
10		удаление динамического объекта dev2	11
11		вызов статического метода showStats	12
12		удаление динамического объекта dev2	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-3.

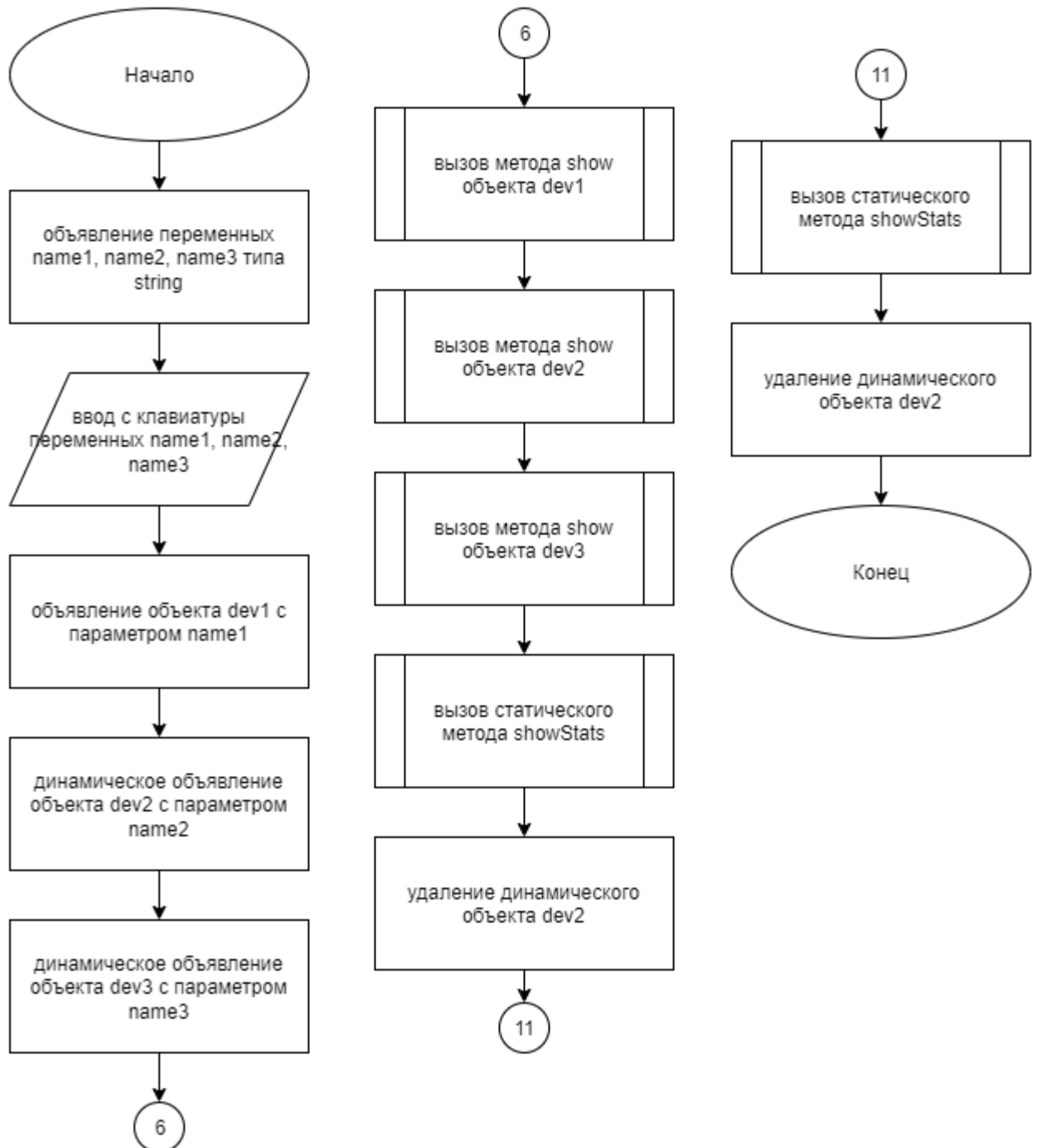


Рисунок 1 – Блок-схема алгоритма

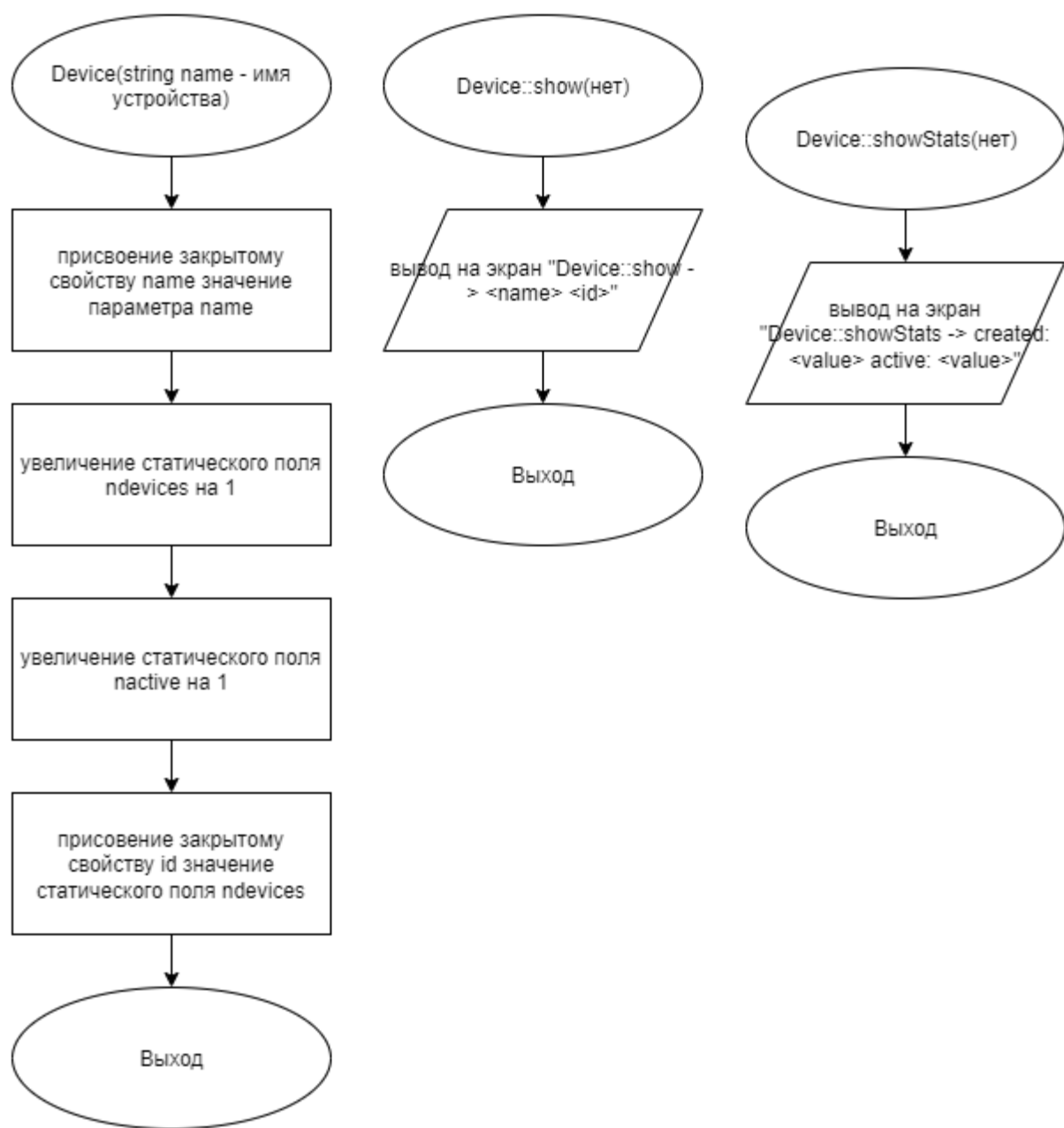


Рисунок 2 – Блок-схема алгоритма



Рисунок 3 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Device.cpp

Листинг 1 – Device.cpp

```
#include "Device.h"
#include <string>

int Device::nactive = 0;
int Device::ndevices = 0;

Device::Device(std::string name)
{
    this->name = name;
    ndevices++;
    nactive++;
    id = ndevices;
}

void Device::show()
{
    printf("Device::show -> %s %d\n", name.c_str(), id);
}

void Device::showStats()
{
    printf("Device::showStats -> created: %d active: %d\n", ndevices,
nactive);
}

Device::~Device()
{
    nactive--;
}
```

5.2 Файл Device.h

Листинг 2 – Device.h

```
#ifndef __DEVICE__H
```

```

#define __DEVICE__H

#include <string>

class Device
{
private:
    std::string name;
    int id;
    static int ndevices;
    static int nactive;

public:
    Device(std::string name);
    void show();
    static void showStats();
    ~Device();
};

#endif

```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>
#include "Device.h"
#include <iostream>

int main()
{
    std::string name1, name2, name3;
    std::cin >> name1 >> name2 >> name3;

    Device dev1(name1);
    Device* dev2 = new Device(name2);
    Device* dev3 = new Device(name3);

    dev1.show();
    dev2->show();
    dev3->show();

    Device::showStats();

    delete dev2;

    Device::showStats();
}

```

```
        delete dev3;  
        return(0);  
    }
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 6.

Таблица 6 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
dev1 dev2 dev3	Device::show -> dev1 1 Device::show -> dev2 2 Device::show -> dev3 3 Device::showStats -> created: 3 active: 3 Device::showStats -> created: 3 active: 2	Device::show -> dev1 1 Device::show -> dev2 2 Device::show -> dev3 3 Device::showStats -> created: 3 active: 3 Device::showStats -> created: 3 active: 2

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoc_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).